Ragasiyam

RAG-Powered Conversational AI Assistant

Project Documentation

Prepared by: Tharun Kumar V

TABLE OF CONTENTS

1. What This Project Does

Ragasiyam is a full-stack conversational AI assistant styled after modern chat assistant interfaces like Claude and ChatGPT. It supports open-ended natural language conversation, and it layers on Retrieval-Augmented Generation (RAG) so users can upload documents and ask questions grounded in their actual content instead of the model's general knowledge alone.

The system was built as a task-driven project rather than a hardened production service. It started from a verbal brief with no written specification, under a two-day deadline. That context matters for interpreting the architecture and feature choices throughout this document, since decisions favored fast, verifiable correctness over exhaustive production hardening.

1.1 Key Capabilities

- Multi-turn conversation with a user-selectable LLM backend (Gemini Flash or Groq/Llama)
- Document upload (PDF/TXT) with automatic chunking, embedding, and retrieval-grounded answering
- An explicit fallback for when no relevant context exists, so the assistant does not fabricate answers from unrelated or absent document content
- Persistent, per-session chat history with search, deletion, and automatically generated chat titles
- Image upload with vision-based description
- Session-scoped data isolation, so documents and retrieval context from one chat session are never visible to another

2. Ideation

The project began with a verbal task brief and no written specification or grading rubric. In the client's own framing, the requested capabilities were a chatbot comparable to Claude or ChatGPT, the ability to chat with uploaded documents, persistent chat history, and continuous availability. Image support was framed as a bonus rather than a hard requirement.

Since no formal spec existed, the first concrete step was translating the verbal brief into an explicit PRD that separated must-have functionality from nice-to-have polish. This distinction shaped the build order directly. Core RAG correctness was prioritized over UI refinement, authentication, and advanced chat management, since those were value add rather than baseline requirements.

2.1 Constraints

- No written specification, so requirements were inferred and documented up front to avoid scope drift
- A firm two-day build window starting from a blank repository
- Free-tier API usage throughout, for both the LLM and embedding providers, which introduced real rate-limit constraints addressed directly during development (see Section 5)

3. Architecture

Ragasiyam uses a decoupled architecture. A Next.js frontend communicates with a FastAPI backend, which coordinates three external managed services rather than hosting any stateful infrastructure itself.

3.1 Request Flow

- The frontend, a Next.js sidebar chat UI, sends requests to the FastAPI backend

- POST /chat embeds the incoming query, runs a similarity search in Qdrant filtered to the active session, and injects any sufficiently relevant chunks into the prompt before calling the selected LLM (Gemini or Groq). If no chunk clears the grounding threshold, the assistant answers without fabricated document context
- POST /upload parses an uploaded document, splits it into overlapping chunks, embeds each chunk through Cohere, and stores the vectors in Qdrant tagged with session and source document metadata
- Both endpoints write activity to MongoDB Atlas, covering chat messages, session metadata, and quota counters

3.2 Component Table

Component	Role
Next.js frontend	Sidebar chat UI, history search, document/image upload, model selector
FastAPI backend	Orchestrates /chat and /upload; routes to LLM and vector store
Gemini Flash / Groq	Selectable LLM providers for chat completion and vision
Cohere Embed	Generates vector embeddings for document chunks and queries
Qdrant (Cloud)	Session-scoped vector similarity search for RAG retrieval
MongoDB Atlas	Persistent chat history, session metadata, quota tracking

3.3 Design Decisions Worth Noting

- **Embeddings run on Cohere rather than the chat LLM's own provider.** The pipeline originally used Gemini for embeddings, which hit free-tier rate limits of 100 requests per minute during real document ingestion. Decoupling the embedding provider from the chat model removed a recurring single point of failure.
- **Retrieval is strictly session-scoped.** A Qdrant payload index and query-time filter on session_id ensure that a document uploaded in one chat is never retrievable from another. This was not the case in an early build and was corrected after being caught during testing, described in Section 5.5.
- **Summarization-style queries bypass similarity ranking.** Broad requests like summarizing a document do not resemble any single chunk closely in embedding space, so they are detected by intent and handled through direct context assembly instead of similarity search.

4. Planning and Execution

Execution was sequenced to avoid the most common failure mode under a hard deadline: building an advanced feature on top of an unverified foundation. Each phase was verified working end to end before the next one began.

- **Phase 1, basic chat loop.** A minimal FastAPI and frontend connection proved that Gemini-backed chat worked end to end, with no persistence or retrieval yet.
- **Phase 2, persistence.** MongoDB Atlas was integrated for chat history, so message storage was proven reliable before the more complex RAG pipeline was layered on top.
- **Phase 3, the RAG pipeline.** File upload, chunking, Cohere embedding, and Qdrant storage were added, along with an explicit grounding check to prevent hallucinated answers when no relevant chunk exists.
- **Phase 4, secondary features.** Multimodal image support, session auto-titling, chat search and deletion, simple guardrails, per-session quota limits, and model switching between Gemini and Groq were added last.

Tooling choices changed more than once in direct response to problems encountered during execution, rather than being fixed in advance. The embedding provider moved from Gemini to Cohere, and the Qdrant deployment target was evaluated between local Docker and Qdrant Cloud, both after real reliability issues surfaced. This is documented in full in Section 5, since the troubleshooting process is as much a part of this project's execution as the final architecture.

5. Obstacles and Resolutions

The build surfaced a series of concrete infrastructure and logic issues. Each was diagnosed from actual error output and stack traces rather than speculative fixes, and each resolution below is documented with its root cause, not just its symptom, since that is what makes a fix verifiable rather than incidental.

5.1 Gemini Free-Tier Rate Limiting

The Gemini API's free tier repeatedly returned 429 ResourceExhausted errors, first on chat completions and then specifically on the embed_content endpoint, which is capped at 100 requests per minute. Because the chat model switcher only affects the completion step, switching to Groq for chat did not resolve ingestion failures, since embeddings were still routed through Gemini regardless of the selected chat model.

- **Resolution.** Provider fallback was added for chat, moving from Gemini to Groq on rate-limit errors, and the embedding provider was migrated entirely to Cohere. Batched embedding calls with retry logic were added, honoring the server-reported retry delay.

5.2 MongoDB Atlas TLS Handshake Failures

Signup and auth requests intermittently failed with a TLS handshake error, an internal alert during the TLS handshake, which is a known issue tied to Node and OpenSSL version mismatches or network-level TLS interference such as antivirus SSL inspection or ISP-level filtering.

- **Resolution.** Atlas IP allowlisting was verified, Node.js was updated to the current LTS release, and VPN and antivirus interference were ruled out through a network-switch test.

5.3 Qdrant Write and Read Timeouts

Document ingestion failed intermittently with write and, separately, read timeouts during vector upsert, despite a trivial single-point upsert succeeding reliably at startup. This isolated the issue to batch-write conditions rather than basic connectivity, authentication, or RAG logic.

- **Resolution.** Upsert batch sizes were reduced, per-batch retry logic was added, and a temporary move to local Qdrant was evaluated as a fallback to remove network variables entirely.

5.4 Missing Qdrant Payload Index

After a session_id filter was added to retrieval queries to fix the leakage described in Section 5.5, Qdrant rejected the filtered search with a 400 error. Qdrant requires an index before a payload field can be used in a query filter, and storing the field alone is not sufficient.

- **Resolution.** A keyword payload index was created on session_id using create_payload_index(), after which filtered retrieval worked as expected.

5.5 Cross-Session Data Leakage

Testing surfaced a case where a question asked in one chat session returned content sourced from a document, a resume, that had been uploaded in a completely different session. The root cause was that the Qdrant similarity search had no session filter at all, so it searched the entire collection across every session and document.

- **Resolution.** An explicit session_id filter was added to the search call, dependent on the index fix above, and verified with a parallel test across two sessions with two different documents.

5.6 Poor Retrieval on Summarization-Style Queries

Specific factual questions retrieved relevant chunks reliably, with similarity scores above 0.55, while broad requests such as asking for a document summary scored low, around 0.25 to 0.29, and were correctly treated as ungrounded.

This reflects a structural limitation of chunk-level similarity search, which does not represent whole-document intent well.

- **Resolution.** Intent detection was added for summarization-style phrasing, bypassing similarity ranking in favor of direct top-chunk or whole-document context assembly for those queries.

5.7 Silent Failures on Long-Running Requests

A subset of chat requests failed with a socket hang-up at the proxy layer with no corresponding backend exception logged, which indicated the request was hanging rather than erroring. The most likely cause was that no explicit timeout was set on the LLM API client call.

- **Resolution.** Explicit timeouts were added on LLM client calls, and the full /chat handler was wrapped in a top-level exception handler with full traceback logging, so every failure path is now diagnosable.

5.8 Generic, Undiagnosable Frontend Error Messages

Multiple distinct backend failures, from Mongo, Qdrant, and Gemini quota limits, were all surfaced to the user as an identical, non-specific error string, which made every failure dependent on manual server-log inspection to diagnose.

- **Resolution.** Backend error responses were standardized to include a specific error field, and frontend error handling was updated to surface that real message instead of a hardcoded fallback string.

6. Features

6.1 Core

- Multi-turn conversational chat with a user-selectable LLM provider (Gemini or Groq)
- Document RAG with session-scoped grounding and an explicit fallback for ungrounded queries
- Persistent, per-session chat history

6.2 Additional

- Image upload with vision-based description
- Session auto-titling, chat search, and chat deletion
- Simple guardrails and per-session and per-user quota limits
- Configurable upload size limits with asynchronous background processing for larger files

7. Limitations and Future Considerations

In the interest of giving an accurate account rather than an inflated one, this system was built and verified within a two-day, task-driven scope, and several of the fixes documented in Section 5 were resolved within hours of submission. The following are known, unaddressed limitations rather than omissions.

- All LLM, embedding, and vector store usage currently runs on free-tier quotas, which are not suitable for sustained multi-user production load without a paid tier or additional caching
- Authentication is functional but has not been hardened. There is no password reset flow, rate-limit tuning is basic, and there is no refresh-token rotation
- Guardrails are a single system-prompt instruction rather than a dedicated moderation pipeline
- Qdrant Cloud showed intermittent write and read latency during heavier ingestion. This is mitigated with retries but not eliminated

These are reasonable trade-offs for the project's actual scope as a task built under a hard deadline, and they are listed here so the system's current state is represented accurately rather than overstated.

8. Output

The result is a working, session-isolated RAG chatbot with a Next.js interface resembling contemporary AI assistant products, including a collapsible history sidebar with search and auto-generated titles, prompt suggestions, document and image upload, and a model selector. Document-grounded questions return contextually accurate answers sourced from the uploaded material, and out-of-scope questions are correctly identified as ungrounded rather than answered with fabricated content.

Beyond the application itself, the development process produced a documented, evidence-based record of common real-world RAG and infrastructure failure modes, including free-tier rate limits, TLS and network flakiness, vector database indexing requirements, cross-session data isolation, and silent request failures, along with the diagnostic process used to resolve each. This is retained in Section 5 as a reference for future work on similar systems.

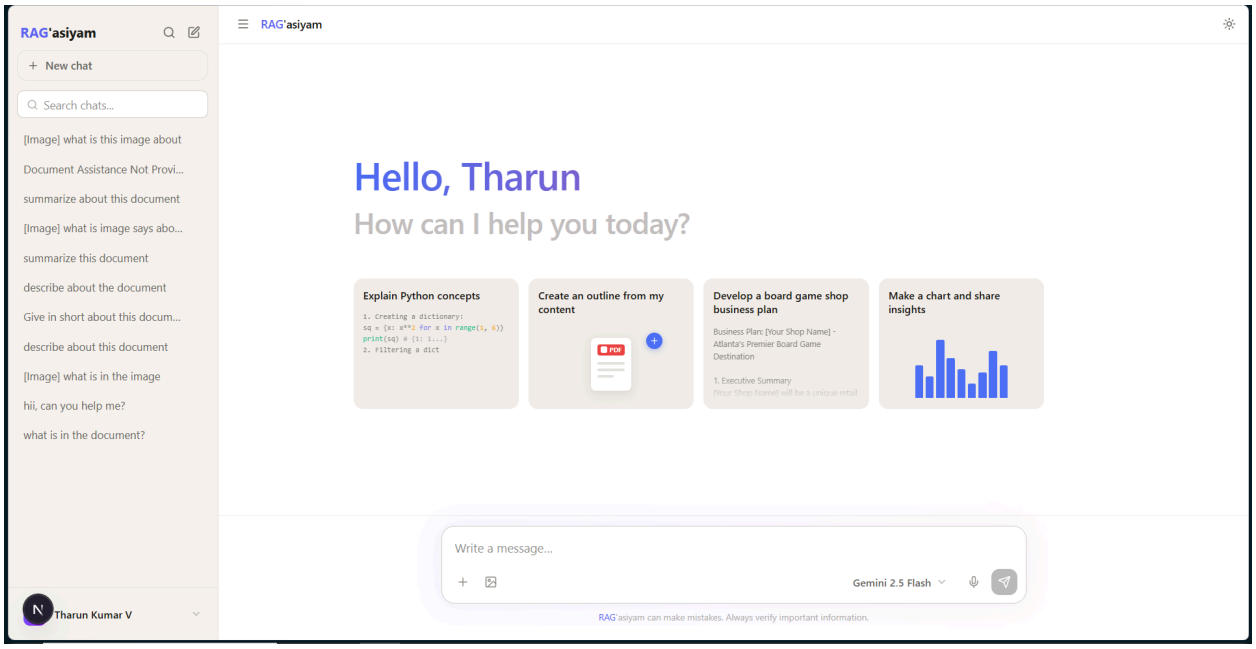

Figure 1: Initial page load of Ragasiyam

End of Document